Criptografía y python

Código de ejemplo para explicar CRC (Cyclic Redundancy Check)

```
# 1. Convertir mensaje en bits (simulación)
def string_to_bits(message):
    return ''.join(format(ord(i), '08b') for i in message)

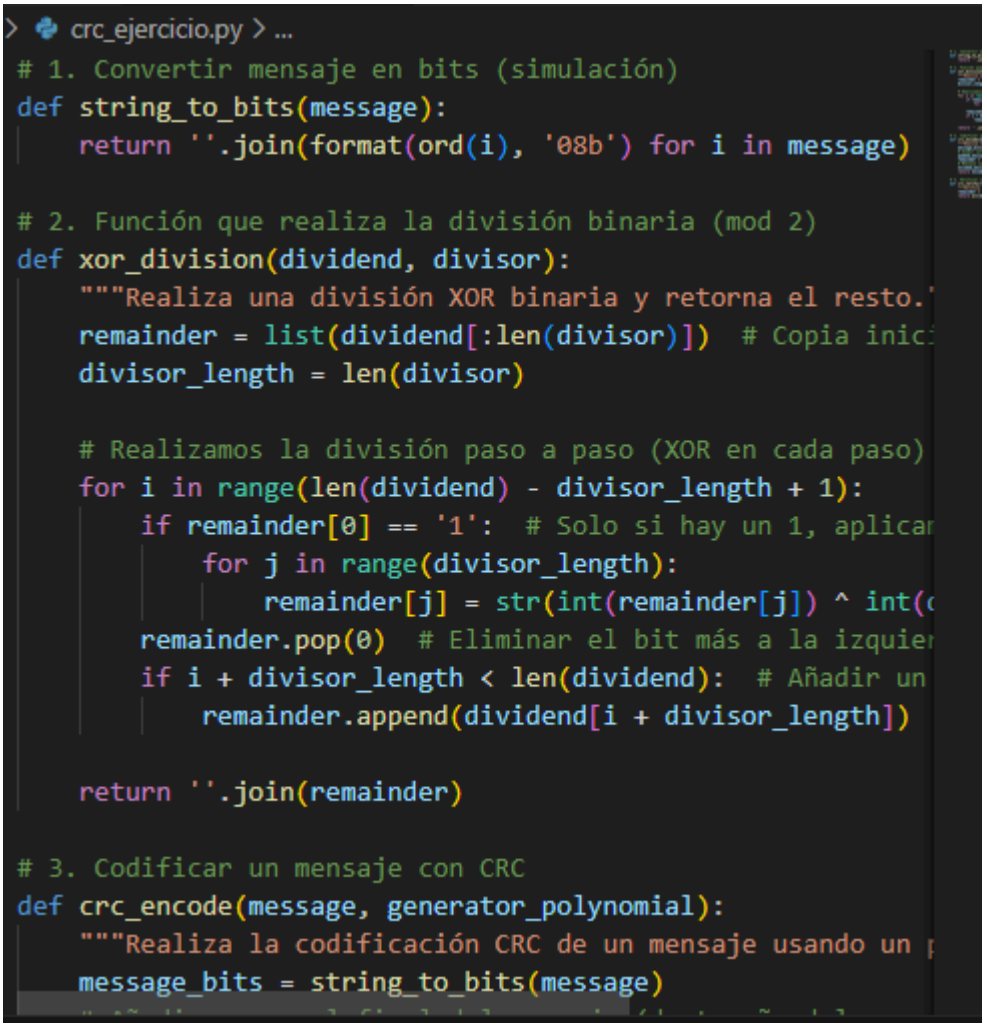
# 2. Función que realiza la división binaria (mod 2)
def xor_division(dividend, divisor):
    """Realiza una división XOR binaria y retorna el resto."""
    remainder = list(dividend[:len(divisor)])  # Copia inicial del dividendo
    divisor_length = len(divisor)

    # Realizamos la división paso a paso (XOR en cada paso)
    for i in range(len(dividend) - divisor_length + 1):
        if remainder[0] == '1':  # Solo si hay un 1, aplicamos XOR
            for j in range(divisor_length):
                remainder[j] = str(int(remainder[j]) ^ int(divisor[j]))
        remainder.pop(0)  # Eliminar el bit más a la izquierda
        if i + divisor_length < len(dividend):  # Añadir un nuevo bit del dividendo
            remainder.append(dividend[i + divisor_length])

    return ''.join(remainder)

# 3. Codificar un mensaje con CRC
def crc_encode(message, generator_polynomial):
    """Realiza la codificación CRC de un mensaje usando un polinomio generador."""
    message_bits = string_to_bits(message)
    # Añadir ceros al final del mensaje (de tamaño del generador - 1)
    padded_message = message_bits + '0' * (len(generator_polynomial) - 1)
    remainder = xor_division(padded_message, generator_polynomial)
    # Mensaje codificado: concatenamos el mensaje original con el residuo CRC
    encoded_message = message_bits + remainder
    return encoded_message, remainder

# 4. Verificar si el mensaje tiene errores utilizando CRC
def crc_verify(received_message, generator_polynomial):
    """Verifica si un mensaje recibido tiene errores aplicando CRC."""
    remainder = xor_division(received_message, generator_polynomial)
    return all(bit == '0' for bit in remainder)  # Si el residuo es cero, no hay error
```




```
# ----------------------------------------
# Ejemplo práctico de uso de CRC
message = "HELLO"
generator_polynomial = '1101'  # Polinomio generador ejemplo (CRC-3)

# Codificamos el mensaje
encoded_message, crc_remainder = crc_encode(message, generator_polynomial)

# Mostrar resultados
print("Mensaje original:", message)
print("Mensaje en bits:", string_to_bits(message))
print("Polinomio generador:", generator_polynomial)
print("Residuo CRC:", crc_remainder)
print("Mensaje codificado:", encoded_message)

# Simulación de recepción del mensaje (correcto)
received_message = encoded_message
print("\nVerificando el mensaje recibido (sin errores)...")
is_valid = crc_verify(received_message, generator_polynomial)
print("¿Mensaje recibido sin errores?:", is_valid)
```

```
# Simulación de recepción del mensaje con error
received_message_with_error = '111' + encoded_message[3:]  # Introducimos un error
print("\nVerificando el mensaje recibido (con errores)...")
is_valid_with_error = crc_verify(received_message_with_error, generator_polynomial)
print("¿Mensaje recibido con errores?:", is_valid_with_error)
```

```
[Running] python -u "c:\Dlore python\python\crc_ejercicio.py"
Mensaje original: Hola
Polinomio generador: 1101
Mensaje codificado CRC: 01001000011011110110110001100001110
Residuo (CRC): 110

Mensaje recibido (sin errores): 01001000011011110110110001100001110
V lido?: True

Mensaje recibido con error: 01011000011011110110110001100001110
V lido?: False

[Done] exited with code=0 in 0.218 seconds
```

Cómo funciona SHA-256

```
import hashlib  # Importamos la librería para usar funciones hash

# 1. Función para obtener el hash de un mensaje usando SHA-256
def calculate_sha256_hash(message):
    # Crear un objeto sha256
    sha256 = hashlib.sha256()
    # Convertir el mensaje a bytes y actualizar el objeto sha256
    sha256.update(message.encode('utf-8'))
    # Devolver el hash en formato hexadecimal
    return sha256.hexdigest()

# 2. Demostración con un mensaje
message = "Hola, este es un ejemplo de hash con SHA-256."
hash_result = calculate_sha256_hash(message)

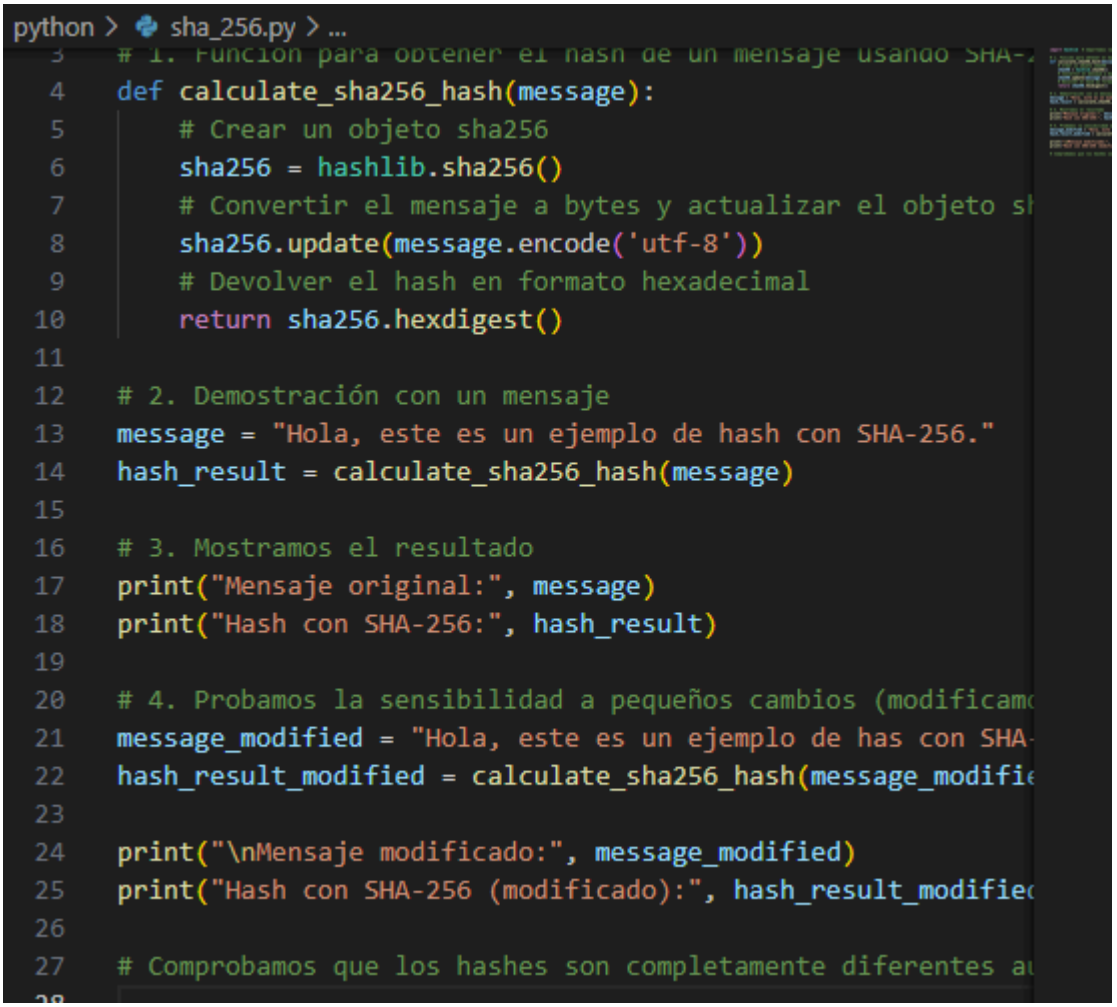
# 3. Mostramos el resultado
print("Mensaje original:", message)
print("Hash con SHA-256:", hash_result)

# 4. Probamos la sensibilidad a pequeños cambios (modificamos una letra)
message_modified = "Hola, este es un ejemplo de has con SHA-256."  # Cambiamos una let
ra
```

hash_result_modified = calculate_sha256_hash(message_modified)

print("\nMensaje modificado:", message_modified)
print("Hash con SHA-256 (modificado):", hash_result_modified)

Comprobamos que los hashes son completamente diferentes aunque solo se cambió una letra




```
[Running] python -u "c:\Dlore python\python\sha_256.py"
Mensaje original: Hola, este es un ejemplo de hash con SHA-256.
Hash con SHA-256:
584cb55df37685b0e4ccfa765b9b4a4eebc6874d5f4d3a8299a964878eb8a010

Mensaje modificado: Hola, este es un ejemplo de has con SHA-256.
Hash con SHA-256 (modificado):
9221e7e950cdef1e0dc4c916730709493d32337f9ceb628a7132da3429736f8e

[Done] exited with code=0 in 0.35 seconds
```

Protocolo Diffie-Hellman

```
import random

# Paso 1: Selección de parámetros públicos (p y g)
p = 23  # Un número primo público
g = 5   # Un generador público (base)

print(f"Parámetros públicos:\n  p = {p}\n  g = {g}\n")

# Paso 2: Cada usuario selecciona su clave privada
# Usuario A elige su clave privada (secreto)
a = random.randint(1, p-1)
print(f"Usuario A selecciona su clave privada (secreto) a = {a}")

# Usuario B elige su clave privada (secreto)
b = random.randint(1, p-1)
print(f"Usuario B selecciona su clave privada (secreto) b = {b}")

# Paso 3: Cálculo de claves públicas
# Usuario A calcula su clave pública
A_pub = (g ** a) % p
print(f"Usuario A calcula su clave pública: A_pub = {A_pub}")

# Usuario B calcula su clave pública
B_pub = (g ** b) % p
print(f"Usuario B calcula su clave pública: B_pub = {B_pub}")

# Paso 4: Intercambio de claves públicas y cálculo de la clave compartida
# Usuario A recibe la clave pública de B y calcula la clave compartida
K_A = (B_pub ** a) % p
print(f"\nUsuario A calcula la clave compartida: K_A = {K_A}")

# Usuario B recibe la clave pública de A y calcula la clave compartida
K_B = (A_pub ** b) % p
print(f"Usuario B calcula la clave compartida: K_B = {K_B}")

# Verificamos que ambas claves compartidas son iguales
if K_A == K_B:
    print(f"\n¡El intercambio de claves ha sido exitoso! Clave compartida: {K_A}")
else:
    print("\nError en el intercambio de claves")
```

python > diffie_hellman.py > ...

```python
import random

# Paso 1: Selección de parámetros públicos (p y g)
p = 23  # Un número primo público
g = 5   # Un generador público (base)

print(f"Parámetros públicos:\n  p = {p}\n  g = {g}\n")

# Paso 2: Cada usuario selecciona su clave privada
# Usuario A elige su clave privada (secreto)
a = random.randint(1, p-1)
print(f"Usuario A selecciona su clave privada (secreto) a =

# Usuario B elige su clave privada (secreto)
b = random.randint(1, p-1)
print(f"Usuario B selecciona su clave privada (secreto) b =

# Paso 3: Cálculo de claves públicas
# Usuario A calcula su clave pública
A_pub = (g ** a) % p
print(f"Usuario A calcula su clave pública: A_pub = {A_pub}"

# Usuario B calcula su clave pública
B_pub = (g ** b) % p
print(f"Usuario B calcula su clave pública: B_pub = {B_pub}"
```

```
[Running] python -u "c:\Dlore python\python\diff
Par?metros p?blicos:
  p = 23
  g = 5

Usuario A selecciona su clave privada (secreto)
Usuario B selecciona su clave privada (secreto)
Usuario A calcula su clave p?blica: A_pub = 9
Usuario B calcula su clave p?blica: B_pub = 21

Usuario A calcula la clave compartida: K_A = 12
Usuario B calcula la clave compartida: K_B = 12

?El intercambio de claves ha sido exitoso! Clav
```

Parámetros públicos

```
p = 23  # Número primo
g = 5   # Generador

# Claves privadas elegidas por Alicia y Benito
a = 6  # Clave privada de Alicia
b = 15  # Clave privada de Benito

# Cálculo de las claves públicas
# Alicia envía A a Benito
A = (g ** a) % p
print(f"Clave pública de Alicia (A): {A}")

# Benito envía B a Alicia
B = (g ** b) % p
print(f"Clave pública de Benito (B): {B}")

# Cálculo de la clave compartida
# Alicia calcula la clave compartida S usando la clave pública de Benito
S_Alicia = (B ** a) % p
print(f"Clave compartida calculada por Alicia (S_Alicia): {S_Alicia}")

# Benito calcula la clave compartida S usando la clave pública de Alicia
S_Benito = (A ** b) % p
print(f"Clave compartida calculada por Benito (S_Benito): {S_Benito}")

# Verificación de que ambas claves compartidas son iguales
if S_Alicia == S_Benito:
    print(f"Éxito, la clave compartida es: {S_Alicia}")
else:
    print("Error, las claves compartidas no coinciden.")
```

python > publicos_parametros.py > ...

```python
p = 23  # Número primo
g = 5   # Generador

# Claves privadas elegidas por Alicia y Benito
a = 6  # Clave privada de Alicia
b = 15  # Clave privada de Benito

# Cálculo de las claves públicas
# Alicia envía A a Benito
A = (g ** a) % p
print(f"Clave pública de Alicia (A): {A}")

# Benito envía B a Alicia
B = (g ** b) % p
print(f"Clave pública de Benito (B): {B}")

# Cálculo de la clave compartida
# Alicia calcula la clave compartida S usando la clave públ
S_Alicia = (B ** a) % p
print(f"Clave compartida calculada por Alicia (S_Alicia): {S

# Benito calcula la clave compartida S usando la clave públ
S_Benito = (A ** b) % p
print(f"Clave compartida calculada por Benito (S_Benito): {S
```

```
[Done] exited with code=0 in 0.235 seconds

[Running] python -u "c:\Dlore python\python\publicos_parametros.py"
Clave p◆blica de Alicia (A): 8
Clave p◆blica de Benito (B): 19
Clave compartida calculada por Alicia (S_Alicia): 2
Clave compartida calculada por Benito (S_Benito): 2
◆xito, la clave compartida es: 2

[Done] exited with code=0 in 0.219 seconds
```

Instalamos Librerias

```
!pip install pycryptodome

# Importamos las bibliotecas necesarias
from Crypto.PublicKey import DSA
from Crypto.Signature import DSS
from Crypto.Hash import SHA256

# 1. Generación de claves

# Alicia genera su par de claves
key_alicia = DSA.generate(2048)

# Benito genera su par de claves
key_benito = DSA.generate(2048)

# 2. Alicia firma un mensaje

# Definimos el mensaje que Alicia quiere enviar
mensaje = b"Hola Benito, este mensaje esta firmado por Alicia."

# Alicia genera el hash del mensaje
hash_mensaje = SHA256.new(mensaje)

# Alicia firma el hash del mensaje usando su clave privada
firmante = DSS.new(key_alicia, 'fips-186-3')
firma = firmante.sign(hash_mensaje)

print("Mensaje enviado por Alicia: ", mensaje)
print("Firma generada por Alicia: ", firma.hex())

# 3. Benito verifica la firma

# Benito recibe el mensaje y la firma
mensaje_recibido = mensaje  # En la vida real, este mensaje sería recibido por la red
firma_recibida = firma  # En la vida real, la firma sería recibida por la red

# Generamos el hash del mensaje recibido por Benito
hash_recibido = SHA256.new(mensaje_recibido)

# Benito verifica la firma usando la clave pública de Alicia
verificador = DSS.new(key_alicia.publickey(), 'fips-186-3')

try:
    verificador.verify(hash_recibido, firma_recibida)
    print("La firma es válida. El mensaje es auténtico y fue enviado por Alicia.")
```

```
except ValueError:
    print("La firma no es válida. El mensaje puede haber sido modificado o no proviene de Ali
cia.")

# 4. Benito responde a Alicia firmando un mensaje

mensaje_benito = b"Hola Alicia, confirmo que he recibido tu mensaje."

# Benito firma su mensaje
hash_mensaje_benito = SHA256.new(mensaje_benito)
firmante_benito = DSS.new(key_benito, 'fips-186-3')
firma_benito = firmante_benito.sign(hash_mensaje_benito)

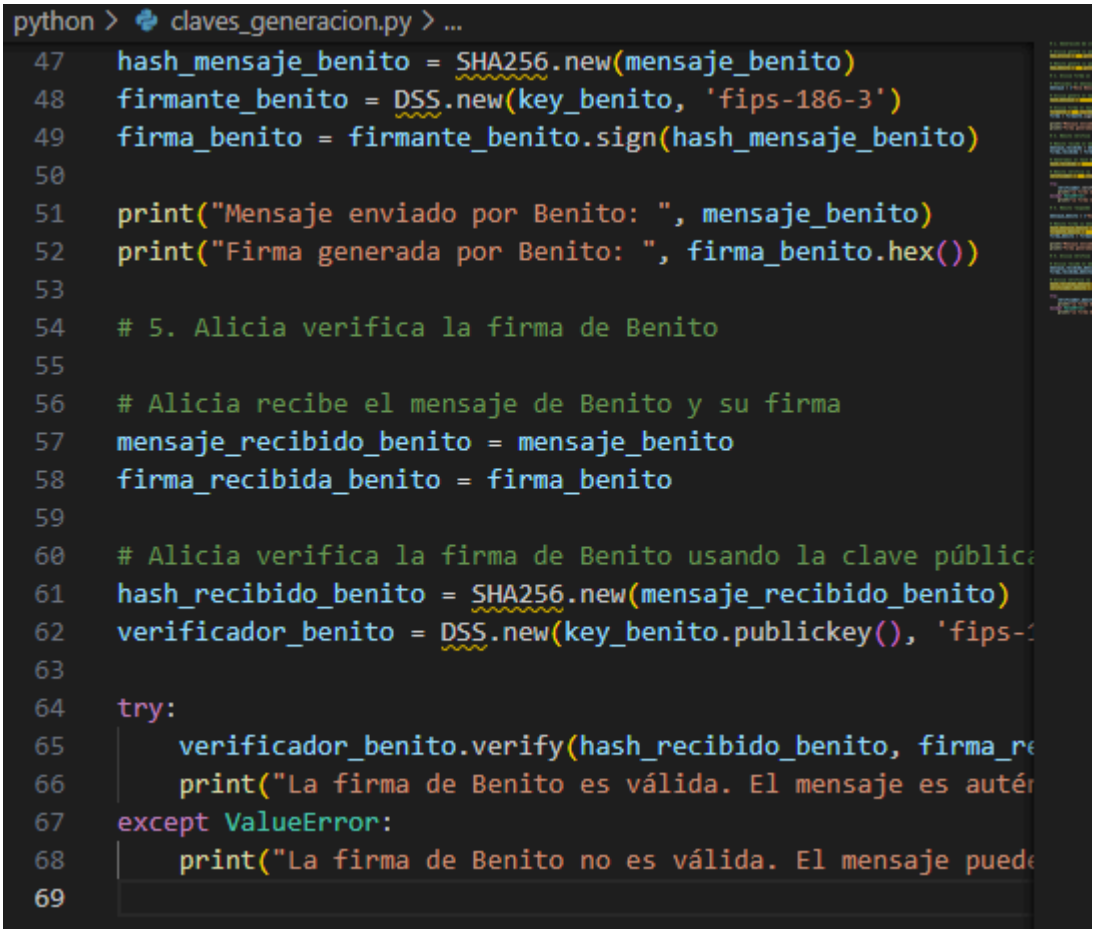
print("Mensaje enviado por Benito: ", mensaje_benito)
print("Firma generada por Benito: ", firma_benito.hex())

# 5. Alicia verifica la firma de Benito

# Alicia recibe el mensaje de Benito y su firma
mensaje_recibido_benito = mensaje_benito
firma_recibida_benito = firma_benito

# Alicia verifica la firma de Benito usando la clave pública de Benito
hash_recibido_benito = SHA256.new(mensaje_recibido_benito)
verificador_benito = DSS.new(key_benito.publickey(), 'fips-186-3')

try:
    verificador_benito.verify(hash_recibido_benito, firma_recibida_benito)
    print("La firma de Benito es válida. El mensaje es auténtico y fue enviado por Benito.")
except ValueError:
    print("La firma de Benito no es válida. El mensaje puede haber sido modificado o no provi
ene de Benito.")
```



```
python > claves_generacion.py > ...
47  hash_mensaje_benito = SHA256.new(mensaje_benito)
48  firmante_benito = DSS.new(key_benito, 'fips-186-3')
49  firma_benito = firmante_benito.sign(hash_mensaje_benito)
50
51  print("Mensaje enviado por Benito: ", mensaje_benito)
52  print("Firma generada por Benito: ", firma_benito.hex())
53
54  # 5. Alicia verifica la firma de Benito
55
56  # Alicia recibe el mensaje de Benito y su firma
57  mensaje_recibido_benito = mensaje_benito
58  firma_recibida_benito = firma_benito
59
60  # Alicia verifica la firma de Benito usando la clave públic
61  hash_recibido_benito = SHA256.new(mensaje_recibido_benito)
62  verificador_benito = DSS.new(key_benito.publickey(), 'fips-
63
64  try:
65      verificador_benito.verify(hash_recibido_benito, firma_re
66      print("La firma de Benito es válida. El mensaje es autén
67  except ValueError:
68      print("La firma de Benito no es válida. El mensaje puede
69
```

Ejemplo en Python - Firma digital con DSA

```
# Paso 1: Cálculo de la clave pública de Alicia
p = 23  # Número primo
q = 11  # Otro primo
h = 2
g = pow(h, (p - 1) // q, p)
x = 10  # Clave privada de Alicia
y = pow(g, x, p)  # Clave pública de Alicia
print("Clave pública de Alicia (y):", y)

# Paso 2: Generación de la firma digital
# Hash del mensaje
H = 5  # Hash simulado del mensaje "Hola"
k = 3  # Número aleatorio
r = pow(g, k, p) % q
print("Valor de r:", r)

# Usamos el algoritmo extendido de Euclides para calcular el inverso de k mod q
def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
```

```
    else:
        g, y, x = extended_gcd(b % a, a)
        return g, x - (b // a) * y, y

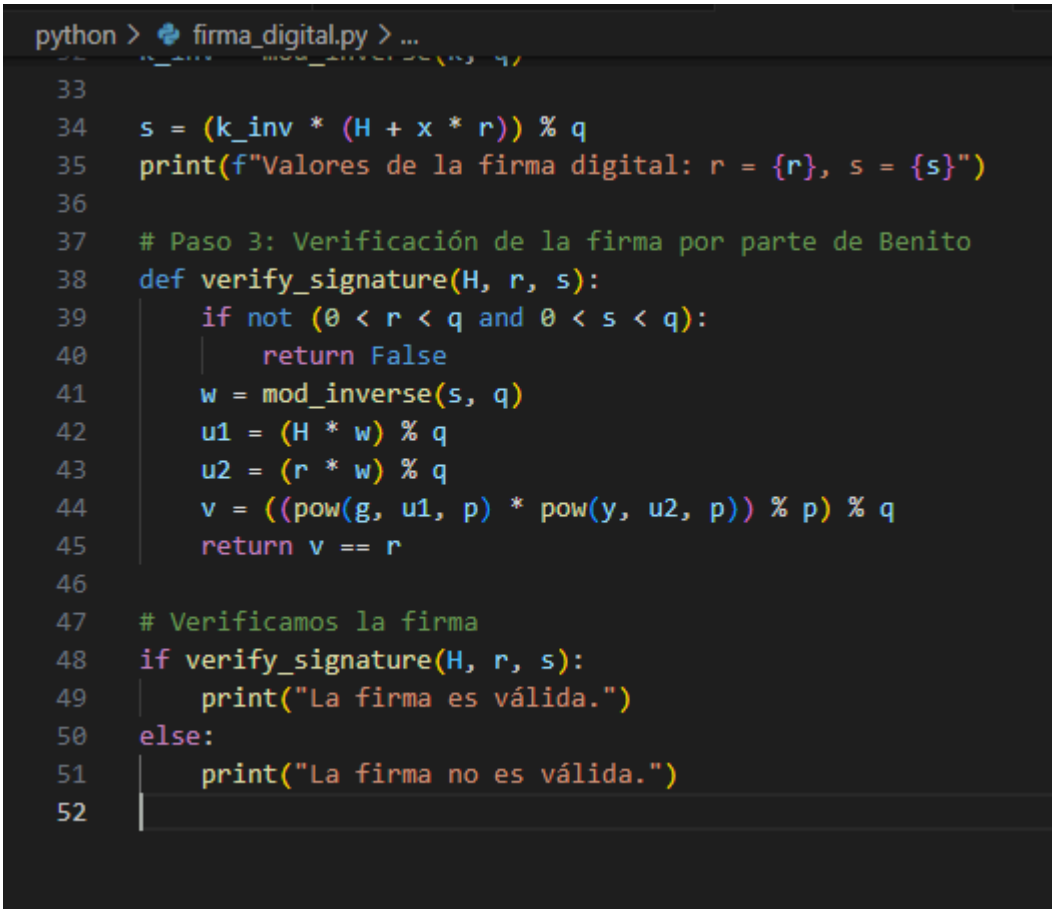
def mod_inverse(a, m):
    g, x, y = extended_gcd(a, m)
    if g != 1:
        raise Exception("No tiene inverso")
    else:
        return x % m

k_inv = mod_inverse(k, q)

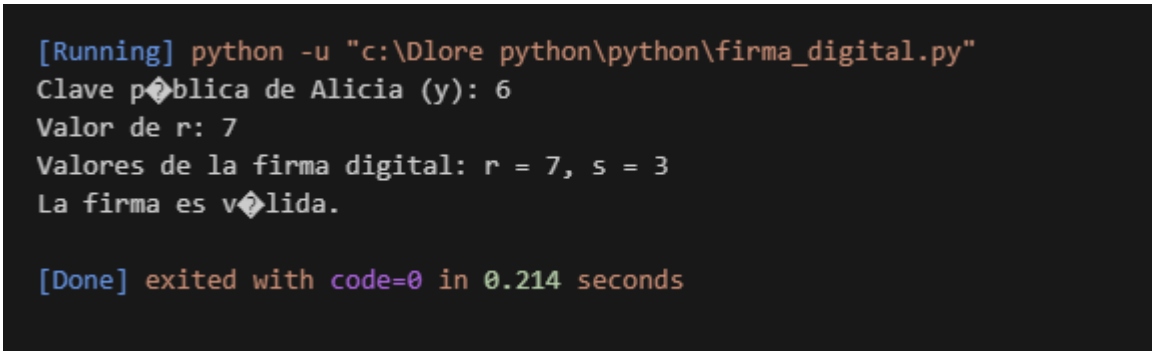
s = (k_inv * (H + x * r)) % q
print(f"Valores de la firma digital: r = {r}, s = {s}")

# Paso 3: Verificación de la firma por parte de Benito
def verify_signature(H, r, s):
    if not (0 < r < q and 0 < s < q):
        return False
    w = mod_inverse(s, q)
    u1 = (H * w) % q
    u2 = (r * w) % q
    v = ((pow(g, u1, p) * pow(y, u2, p)) % p) % q
    return v == r

# Verificamos la firma
if verify_signature(H, r, s):
    print("La firma es válida.")
else:
    print("La firma no es válida.")
```



```
python > firma_digital.py > ...
33
34  s = (k_inv * (H + x * r)) % q
35  print(f"Valores de la firma digital: r = {r}, s = {s}")
36
37  # Paso 3: Verificación de la firma por parte de Benito
38  def verify_signature(H, r, s):
39      if not (0 < r < q and 0 < s < q):
40          return False
41      w = mod_inverse(s, q)
42      u1 = (H * w) % q
43      u2 = (r * w) % q
44      v = ((pow(g, u1, p) * pow(y, u2, p)) % p) % q
45      return v == r
46
47  # Verificamos la firma
48  if verify_signature(H, r, s):
49      print("La firma es válida.")
50  else:
51      print("La firma no es válida.")
52
```



```
[Running] python -u "c:\Dlore python\python\firma_digital.py"
Clave p□blica de Alicia (y): 6
Valor de r: 7
Valores de la firma digital: r = 7, s = 3
La firma es v□lida.

[Done] exited with code=0 in 0.214 seconds
```

Funcionamiento del proceso en python

```
import random
from sympy import mod_inverse

# Función para verificar si un número es primo
def es_primo(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

```
# Función para generar un número primo aleatorio en un rango
def generar_primo(rango_min, rango_max):
    primo = random.randint(rango_min, rango_max)
    while not es_primo(primo):
        primo = random.randint(rango_min, rango_max)
    return primo

# Paso 1: Generar dos primos pequeños p y q
p = generar_primo(50, 100)
q = generar_primo(50, 100)
print(f"Primos generados: p = {p}, q = {q}")

# Paso 2: Calcular n y phi(n)
n = p * q
phi_n = (p - 1) * (q - 1)
print(f"n = {n}, phi(n) = {phi_n}")

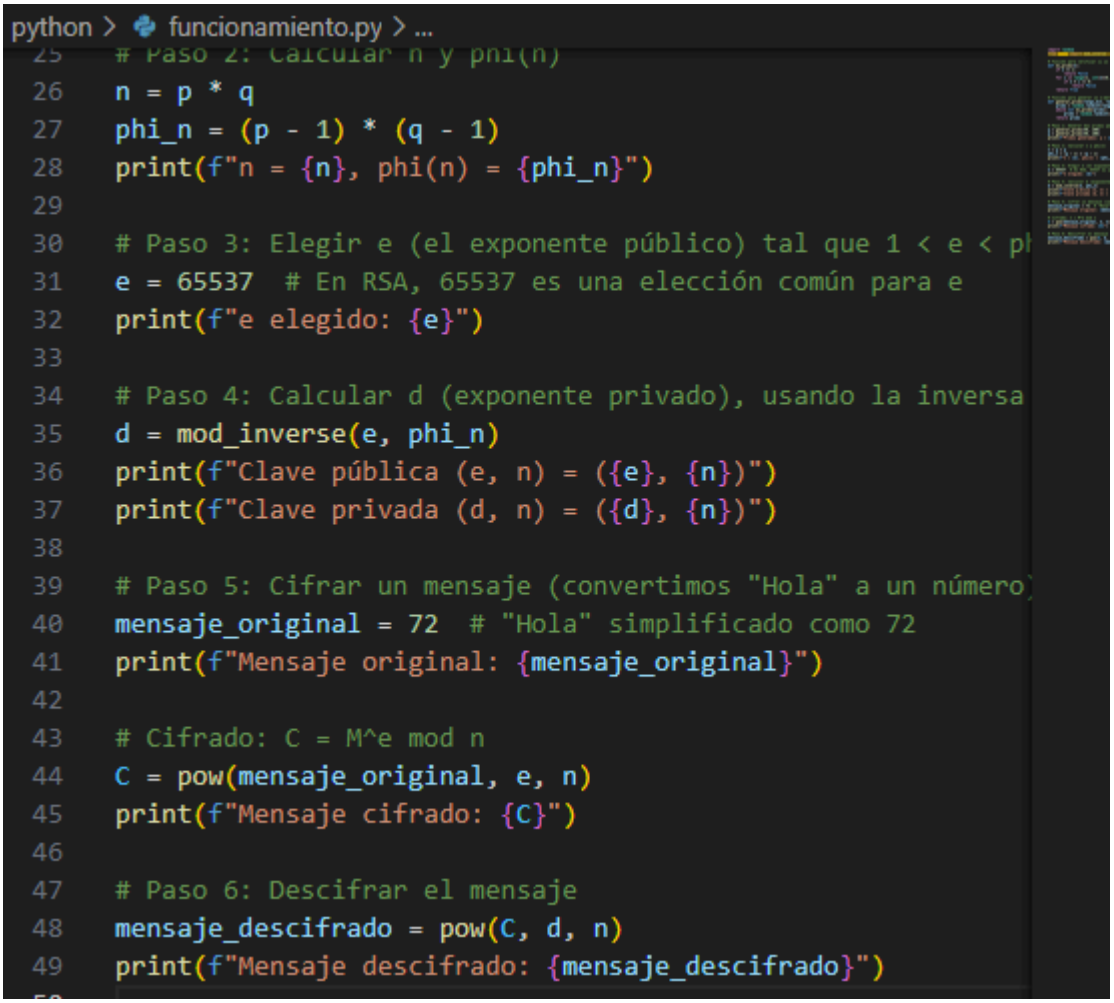
# Paso 3: Elegir e (el exponente público) tal que 1 < e < phi(n) y coprimo con phi(n)
e = 65537  # En RSA, 65537 es una elección común para e
print(f"e elegido: {e}")

# Paso 4: Calcular d (exponente privado), usando la inversa modular de e mod phi(n)
d = mod_inverse(e, phi_n)
print(f"Clave pública (e, n) = ({e}, {n})")
print(f"Clave privada (d, n) = ({d}, {n})")

# Paso 5: Cifrar un mensaje (convertimos "Hola" a un número)
mensaje_original = 72  # "Hola" simplificado como 72
print(f"Mensaje original: {mensaje_original}")

# Cifrado: C = M^e mod n
C = pow(mensaje_original, e, n)
print(f"Mensaje cifrado: {C}")

# Paso 6: Descifrar el mensaje
mensaje_descifrado = pow(C, d, n)
print(f"Mensaje descifrado: {mensaje_descifrado}")
```



Funcionamiento de SHA-256-A

```
import hashlib
import binascii

# Mensaje original
mensaje = "Hola, mundo"

# Convertir el mensaje a bytes
mensaje_bytes = mensaje.encode()

# Mostrar el mensaje en bytes
print("Mensaje en bytes:")
print(mensaje_bytes)

# Fase 1: Preparación del mensaje (padding)
def padding(mensaje_bytes):
    mensaje_bin = ''.join(format(byte, '08b') for byte in mensaje_bytes)
    mensaje_len = len(mensaje_bin)
```

```
    # Agregar '1' al final del mensaje
    mensaje_bin += '1'

    # Rellenar con ceros hasta que el largo sea congruente con 448 mod 512
    while len(mensaje_bin) % 512 != 448:
        mensaje_bin += '0'

    # Agregar la longitud original del mensaje al final, en 64 bits
    mensaje_bin += format(mensaje_len, '064b')

    return mensaje_bin

mensaje_padded = padding(mensaje_bytes)

# Mostrar el mensaje después del padding
print("\nMensaje después del padding:")
print(mensaje_padded)

# Fase 2: Hash utilizando hashlib para obtener el hash final (con todas las rondas internas)
hash_sha256 = hashlib.sha256(mensaje_bytes)

# Debido a la complejidad del algoritmo SHA-256, algunas de las etapas no se pueden impl
ementar
# sin una reescritura detallada del algoritmo. Estas etapas incluyen la expansión del mensaj
e
# y las rondas de compresión, que involucran operaciones matemáticas avanzadas (rotacio
nes, mezclas bit a bit, etc.).

# Obtener el valor del hash en formato hexadecimal
hash_hex = hash_sha256.hexdigest()

# Mostrar el resultado del hash
print("\nHash SHA-256 final (resultado):")
print(hash_hex)
```

python > sha256f.py > ...

```python
def padding(mensaje_bytes):
    # Agregar la longitud original del mensaje al final, en
    mensaje_bin += format(mensaje_len, '064b')

    return mensaje_bin

mensaje_padded = padding(mensaje_bytes)

# Mostrar el mensaje después del padding
print("\nMensaje después del padding:")
print(mensaje_padded)

# Fase 2: Hash utilizando hashlib para obtener el hash final
hash_sha256 = hashlib.sha256(mensaje_bytes)

# Debido a la complejidad del algoritmo SHA-256, algunas de
# sin una reescritura detallada del algoritmo. Estas etapas
# y las rondas de compresión, que involucran operaciones mat

# Obtener el valor del hash en formato hexadecimal
hash_hex = hash_sha256.hexdigest()

# Mostrar el resultado del hash
print("\nHash SHA-256 final (resultado):")
print(hash_hex)
```

```
[Running] python -u "c:\Dlore python\python\sha256f.py"
Mensaje en bytes:
b'Hola, mundo'

Mensaje despu◆s del padding:
0100100001101111011011000110000100101100001000000110110101110101011011100110
1001000110111110000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000001011000

Hash SHA-256 final (resultado):
02e1557c91cd1834cce0b9a824283e1cedf497ed0037338a284aaa70b68b536e

[Done] exited with code=0 in 0.223 seconds
```